//cpu

```
//fcfs
#include <iostream>
using namespace std;
int main() {
 int n, head, size, seek = 0;
 cout << "Enter number of requests: ";
 cin >> n;
 int req[n];
 cout << "Enter requests: ";
 for(int i=0;i<n;i++)
  cin >> req[i];
 cout << "Enter initial head: ";
 cin >> head;
 cout << "Enter disk size: ";
 cin >> size;
 // sorting
 for(int i=0;i<n-1;i++)
  for(int j=0;j<n-i-1;j++)
   if(req[j] > req[j+1]) {
    int t = req[j];
    req[j] = req[j+1];
    req[j+1] = t; }
 int pos = 0;
 while(pos < n && req[pos] < head)
  pos++;
 // move right
 for(int i=pos;i<n;i++) {
  seek += (head > req[i]) ?
head-req[i] : req[i]-head;
  head = req[i]; }
 seek += (size-1) - head;
 // jump to start
 seek += size-1;
 head = 0;
 // continue right
 for(int i=0;i<pos;i++) {
  seek += (head > req[i]) ?
head-req[i] : req[i]-head;
  head = req[i]; }
 cout << "Total Seek Time = " << seek;
 return 0; }
```

//sjf

```
//sjf preemptive
#include<iostream>
using namespace std;
int main(){
 int n;
 cin>>n;
 int at[20],bt[20],rt[20],ct[20],wt[20],tat[20];
 for(int i=0;i<n;i++){
  cin>>at[i]>>bt[i];
  rt[i]=bt[i]; }
 int time=0,completed=0;
 float awt=0,atat=0;
 while(completed<n){
  int idx=-1,minbt=9999;
  for(int i=0;i<n;i++){
   if(at[i]<=time && rt[i]>0 && rt[i]<minbt){
    minbt=rt[i];
    idx=i; } }
  if(idx!=-1){
   rt[idx]--;
   time++;
   if(rt[idx]==0){
    ct[idx]=time;
    tat[idx]=ct[idx]-at[idx];
    wt[idx]=tat[idx]-bt[idx];
    awt+=wt[idx];
    atat+=tat[idx];
    completed++; }}
  else time++; }
 cout<<"PID AT BT CT TAT WT\n";
 for(int i=0;i<n;i++)
  cout<<i+1<<" "<<at[i]<<" "<<bt[i]<<" "
   <<ct[i]<<" "<<tat[i]<<" "<<wt[i]<<"\n";
 cout<<"\nAvg WT="<<awt/n;
 cout<<"\nAvg TAT="<<atat/n; return 0; }
```

```
//non premptive
#include<iostream>
using namespace std;
int main(){
 int n; cin>>n;
 int at[20],bt[20],pr[20],ct[20],
 wt[20],tat[20],done[20]={0};
 for(int i=0;i<n;i++)
  cin>>at[i]>>bt[i]>>pr[i];
 int time=0,completed=0;
 float awt=0,atat=0;
 while(completed<n){ int idx=-1,minp=9999;
  for(int i=0;i<n;i++){
   if(at[i]<=time && !done[i] && pr[i]<minp){
    minp=pr[i]; idx=i; }}
  if(idx!=-1){
   time+=bt[idx]; ct[idx]=time;
   tat[idx]=ct[idx]-at[idx];
   wt[idx]=tat[idx]-bt[idx];
   awt+=wt[idx]; atat+=tat[idx];
   done[idx]=1; completed++; }
  else time++; }
 cout<<"PID AT BT PR CT TAT WT\n";
 for(int i=0;i<n;i++)
  cout<<i+1<<" "<<at[i]<<" "<<bt[i]<<" "<<pr[i]<<" "
   <<ct[i]<<" "<<tat[i]<<" "<<wt[i]<<"\n";
 cout<<"\nAvg WT="<<awt/n; cout<<"\nAvg TAT="<<atat/n; return 0; }
```

//round robin

```
#include<iostream>
using namespace std;
int main(){
 int n,tq;
 cin>>n;
 int at[20],bt[20],rt[20],ct[20],
wt[20],tat[20];
 for(int i=0;i<n;i++){
  cin>>at[i]>>bt[i];
  rt[i]=bt[i]; }
 cin>>tq;
 int time=0,done=0;
 float awt=0,atat=0;
 while(done<n){
  bool f=false;
  for(int i=0;i<n;i++){
   if(at[i]<=time && rt[i]>0){
    f=true;
    if(rt[i]>tq){
     time+=tq;
     rt[i]-=tq; }
    else{
     time+=rt[i];
     rt[i]=0;
     ct[i]=time;
     tat[i]=ct[i]-at[i];
     wt[i]=tat[i]-bt[i];
     awt+=wt[i];
     atat+=tat[i];
     done++; } } }
 if(!f) time++; }
cout<<"PID AT BT CT TAT WT\n";
for(int i=0;i<n;i++)
 cout<<i+1<<" "<<at[i]<<" "<<bt[i]<<" "
  <<ct[i]<<" "<<tat[i]<<" "<<wt[i]<<"\n";
cout<<"\nAvg WT="<<awt/n;
cout<<"\nAvg TAT="<<atat/n; return 0; }
```

//non premptive priority

```
#include<iostream>
using namespace std;
int main(){
 int n;
 cin>>n;
 int at[20],bt[20],pr[20],ct[20],
 wt[20],tat[20],done[20]={0};
 for(int i=0;i<n;i++)
  cin>>at[i]>>bt[i]>>pr[i];
 int time=0,completed=0;
 float awt=0,atat=0;
 while(completed<n){
  int idx=-1,minp=9999;
  for(int i=0;i<n;i++){
   if(at[i]<=time && !done[i] && pr[i]<minp){
    minp=pr[i];
    idx=i; }}
  if(idx!=-1){
   time+=bt[idx];
   ct[idx]=time;
   tat[idx]=ct[idx]-at[idx];
   wt[idx]=tat[idx]-bt[idx];
   awt+=wt[idx];
   atat+=tat[idx];
   done[idx]=1;
   completed++; }
  else time++; }
 cout<<"PID AT BT PR CT TAT WT\n";
 for(int i=0;i<n;i++)
  cout<<i+1<<" "<<at[i]<<" "<<bt[i]<<" "<<pr[i]<<" "
   <<ct[i]<<" "<<tat[i]<<" "<<wt[i]<<"\n";
 cout<<"\nAvg WT="<<awt/n;
 cout<<"\nAvg TAT="<<atat/n; return 0; }
```

//pre priority

```
#include<iostream>
using namespace std;
int main(){
 int n;
 cin>>n;
 int at[20],bt[20],rt[20],
 pr[20],ct[20],wt[20],tat[20];
 for(int i=0;i<n;i++){
  cin>>at[i]>>bt[i]>>pr[i];
  rt[i]=bt[i]; }
 int time=0,completed=0;
 float awt=0,atat=0;
 while(completed<n){
  int idx=-1,minp=9999;
  for(int i=0;i<n;i++){
   if(at[i]<=time && rt[i]>0 && pr[i]<minp){
    minp=pr[i];
    idx=i; }  }
  if(idx!=-1){
   rt[idx]--;
   time++;
   if(rt[idx]==0){
    ct[idx]=time;
    tat[idx]=ct[idx]-at[idx];
    wt[idx]=tat[idx]-bt[idx];
    awt+=wt[idx];
    atat+=tat[idx];
    completed++; }   }
  else time++; }
 cout<<"PID AT BT PR CT TAT WT\n";
 for(int i=0;i<n;i++)
  cout<<i+1<<" "<<at[i]<<" "<<bt[i]<<" "<<pr[i]<<" "
   <<ct[i]<<" "<<tat[i]<<" "<<wt[i]<<"\n";
 cout<<"\nAvg WT="<<awt/n;
 cout<<"\nAvg TAT="<<atat/n; return 0; }
```

//c-scan

```
#include <iostream>
using namespace std;
int main() {
 int n, head, size, seek = 0;
 cout << "Enter number of requests: ";
 cin >> n;
 int req[n];
 cout << "Enter requests: ";
 for(int i=0;i<n;i++)
  cin >> req[i];
 cout << "Enter initial head: ";
 cin >> head;
 cout << "Enter disk size: ";
 cin >> size;
 // sorting
 for(int i=0;i<n-1;i++)
  for(int j=0;j<n-i-1;j++)
   if(req[j] > req[j+1]) {
    int t = req[j];
    req[j] = req[j+1];
    req[j+1] = t; }
 int pos = 0;
 while(pos < n && req[pos] < head)
  pos++;
 // move right
 for(int i=pos;i<n;i++) {
  seek += (head > req[i]) ?
head-req[i] : req[i]-head;
  head = req[i]; }
 seek += (size-1) - head;
 // jump to start
 seek += size-1;
 head = 0;
 // continue right
 for(int i=0;i<pos;i++) {
  seek += (head > req[i]) ?
head-req[i] : req[i]-head;
  head = req[i]; }
 cout << "Total Seek Time = " << seek;
 return 0; }
```

//scan

```
#include <iostream>
using namespace std;
int main() {
 int n, head, size, seek = 0;
 cout << "Enter number of requests: ";
 cin >> n;
 int req[n];
 cout << "Enter requests: ";
 for(int i=0;i<n;i++)
  cin >> req[i];
 cout << "Enter initial head: ";
 cin >> head;
 cout << "Enter disk size: ";
 cin >> size;
 // sorting
 for(int i=0;i<n-1;i++)
  for(int j=0;j<n-i-1;j++)
   if(req[j] > req[j+1]) {
    int t = req[j];
    req[j] = req[j+1];
    req[j+1] = t; }
 int pos = 0;
 while(pos < n && req[pos] < head)
  pos++;
 // move right
 for(int i=pos;i<n;i++) {
  seek += (head > req[i]) ?
head-req[i] : req[i]-head;
  head = req[i]; }
 seek += (size-1) - head;
 head = size-1;
 // move left
 for(int i=pos-1;i>=0;i--) {
  seek += (head > req[i]) ?
head-req[i] : req[i]-head;
  head = req[i]; }
 cout << "Total Seek Time = "
 << seek; return 0; }
```

//consumer producer problem

```
#include<iostream>
#include<thread>
#include<mutex>
#include<semaphore>
using namespace std;
int buffer[5], in=0, out=0;
counting_semaphore<5> empty(5);
counting_semaphore<5> full(0);
mutex mtx;
void producer(int id){
 for(int i=1;i<=5;i++){
  empty.acquire();
  mtx.lock();
  buffer[in]=i;
  cout<<"Producer "<<id<<" produced "<<i<<endl;
  in=(in+1)%5;
  mtx.unlock();
  full.release(); }}
void consumer(int id){
 for(int i=1;i<=5;i++){
  full.acquire();
  mtx.lock();
  cout<<"Consumer "<<id<<" consumed "<<buffer[out]<<endl;
  out=(out+1)%5;
  mtx.unlock();
  empty.release(); }}
int main(){
 thread p1(producer,1), c1(consumer,1);
 p1.join();
 c1.join();
 return 0; }
```

Dining philosopher

```
#include<iostream>
#include<thread>
#include<mutex>
#include<semaphore>
using namespace std;
const int N=5;
mutex forks[N];
binary_semaphore room(4);
void philosopher(int id){
 room.acquire();
 forks[id].lock();
 forks[(id+1)%N].lock();
 cout<<"Philosopher "<<id<<" is eating"<<endl;
 forks[id].unlock();
 forks[(id+1)%N].unlock();
 room.release();}
int main(){
 thread p[N];
 for(int i=0;i<N;i++)
  p[i]=thread(philosopher,i);
 for(int i=0;i<N;i++)
  p[i].join();
 return 0; }
```

//reader writer

```
#include<iostream>
#include<thread>
#include<mutex>
#include<semaphore>
using namespace std;
int readcount=0;
mutex mtx;
binary_semaphore wrt(1);
void reader(int id){
 mtx.lock();
 readcount++;
 if(readcount==1)
  wrt.acquire();
 mtx.unlock();
 cout<<"Reader "<<id<<" is reading"<<endl;
 mtx.lock();
 readcount--;
 if(readcount==0)
  wrt.release();
 mtx.unlock();}
void writer(int id){
 wrt.acquire();
 cout<<"Writer "<<id<<" is writing"<<endl;
 wrt.release();}
int main(){
 thread r1(reader,1), r2(reader,2), w1(writer,1);
 r1.join();
 r2.join();
 w1.join();
 return 0; }
```